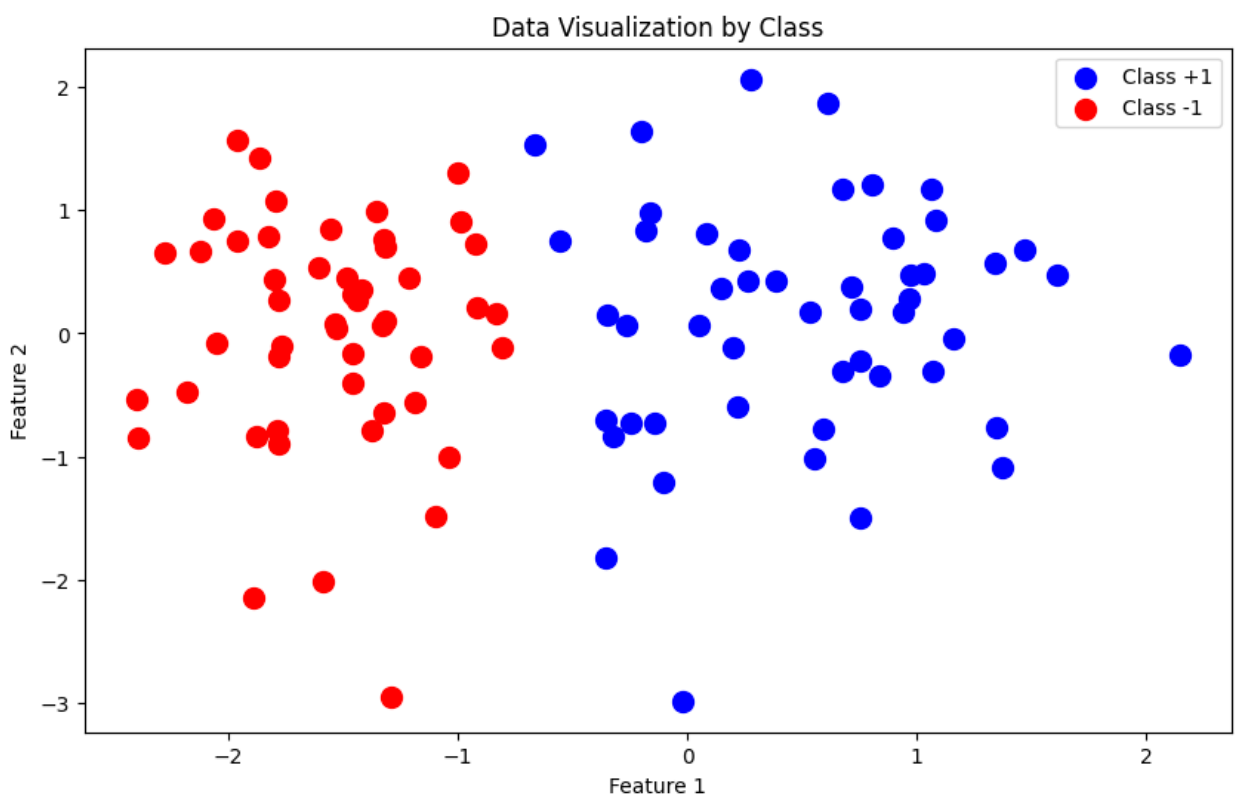
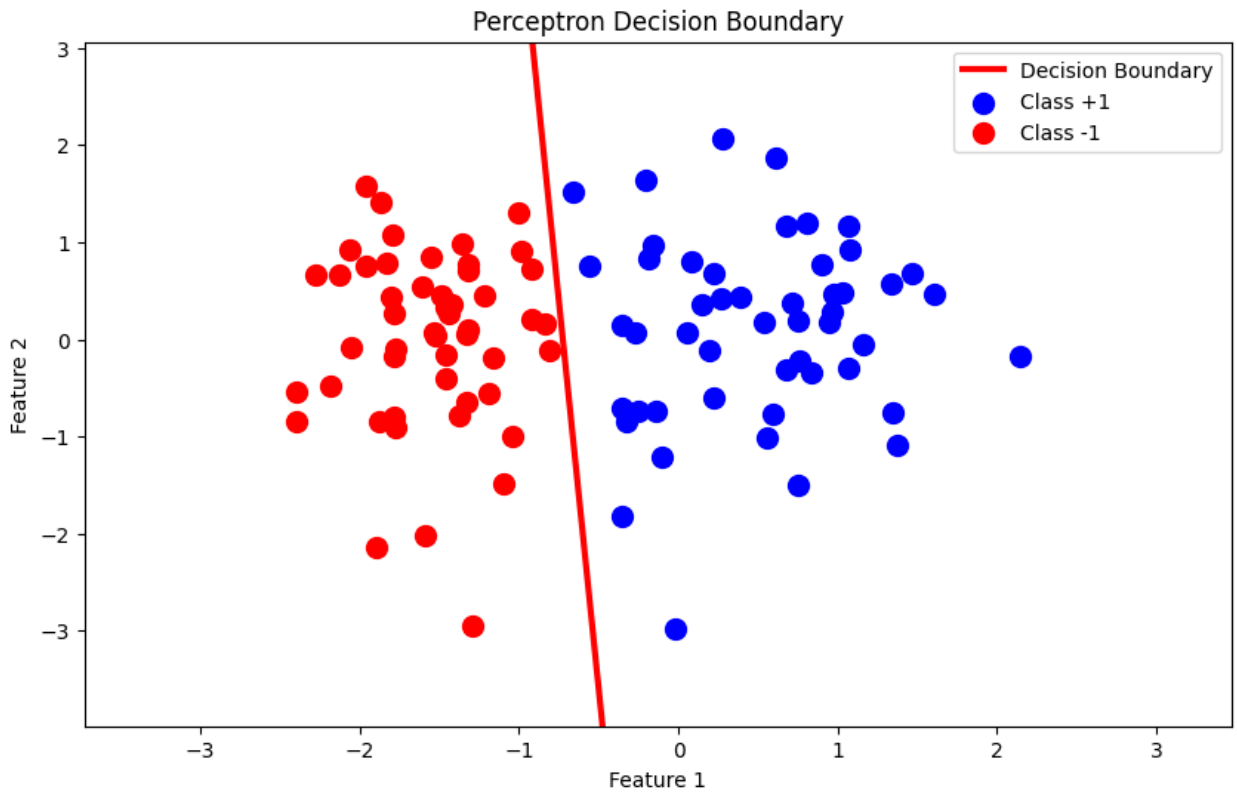
```python
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification
X,y = make_classification(n_samples=100, n_features=2,
n_informative=1,n_redundant=0,
                           n_classes=2, n_clusters_per_class=1,
random_state=41, hypercube=False, class_sep=10)
# Convert y from {0, 1} to {-1, 1} to match the reference perceptron
logic
y = np.where(y == 0, -1, 1)

import matplotlib.pyplot as plt
plt.figure(figsize=(10,6))
plt.scatter(X[y==1, 0], X[y==1, 1], color='blue', label='Class +1',
s=100)
plt.scatter(X[y==0, 0], X[y==0, 1], color='red', label='Class -1',
s=100)
plt.legend()
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Data Visualization by Class')
plt.show()
```



```python
import numpy as np

def perceptron(X, y, iterations=100, lr=0.1):
    w = np.zeros(X.shape[1])
    b = 0

    for _ in range(iterations):
        for i in range(len(X)):
            # The condition y[i]*(np.dot(w, X[i]) + b) <= 0 means
misclassified
            if y[i]*(np.dot(w, X[i]) + b) <= 0:
                w = w + lr*y[i]*X[i]
                b = b + lr*y[i]
    return w, b

# Train the perceptron using the new function signature
w, b = perceptron(X, y, iterations=100, lr=0.1)
print("Learned Weights (w):", w)
print("Learned Bias (b):", b)
# Plot decision boundary
x_vals = np.linspace(X[:,0].min() - 1, X[:,0].max() + 1, 100)
# Decision boundary: w[0]*x_vals + w[1]*y_vals + b = 0 => y_vals = -
(w[0]*x_vals + b)/w[1]
y_vals = -(w[0]*x_vals + b)/w[1]
```

```
Learned Weights (w): [0.27594296 0.01727671]
Learned Bias (b): 0.2
```

```python
plt.figure(figsize=(10,6))
plt.plot(x_vals, y_vals, color='red', linewidth=3, label='Decision
Boundary')

# Plot data points using the updated y labels
plt.scatter(X[y==1, 0], X[y==1, 1], color='blue', label='Class +1',
s=100)
plt.scatter(X[y==-1, 0], X[y==-1, 1], color='red', label='Class -1',
s=100)
plt.legend()
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Perceptron Decision Boundary')
plt.ylim(X[:,1].min() - 1, X[:,1].max() + 1) # Adjust y-limits
dynamically
plt.show()
```



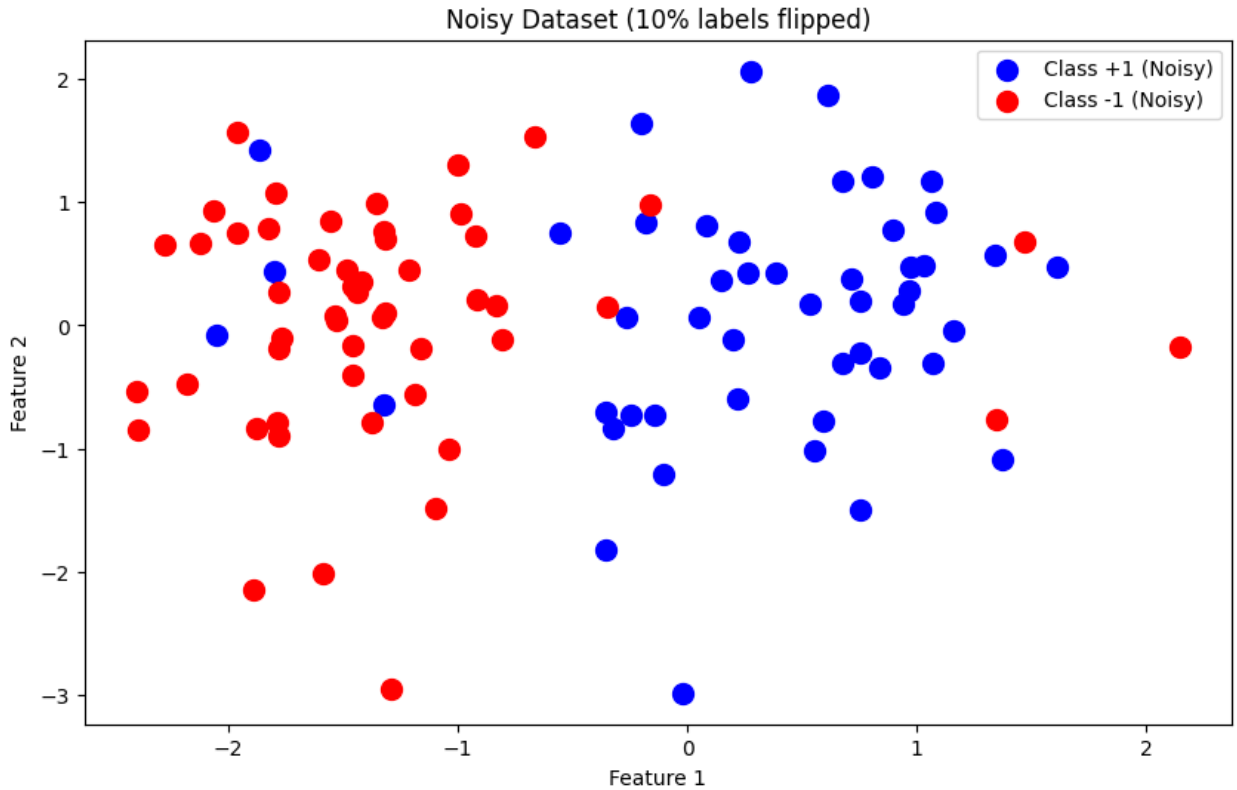

```
# Calculate and print accuracy
pred = np.sign(np.dot(X,w) + b)
accuracy = np.mean(pred == y)
print("Accuracy:", accuracy)
```

```
Accuracy: 1.0
```

Noise added

```
n_samples = len(X)
noise_indices = np.random.choice(n_samples, size=int(0.1 * n_samples),
replace=False)
y_noisy = y.copy()
y_noisy[noise_indices] *= -1

# Plot noisy dataset
plt.figure(figsize=(10,6))
plt.scatter(X[y_noisy==1, 0], X[y_noisy==1, 1], color='blue',
label='Class +1 (Noisy)', s=100)
plt.scatter(X[y_noisy==-1, 0], X[y_noisy==-1, 1], color='red',
label='Class -1 (Noisy)', s=100)
plt.title("Noisy Dataset (10% labels flipped)")
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
```

```
plt.legend()
plt.show()
```



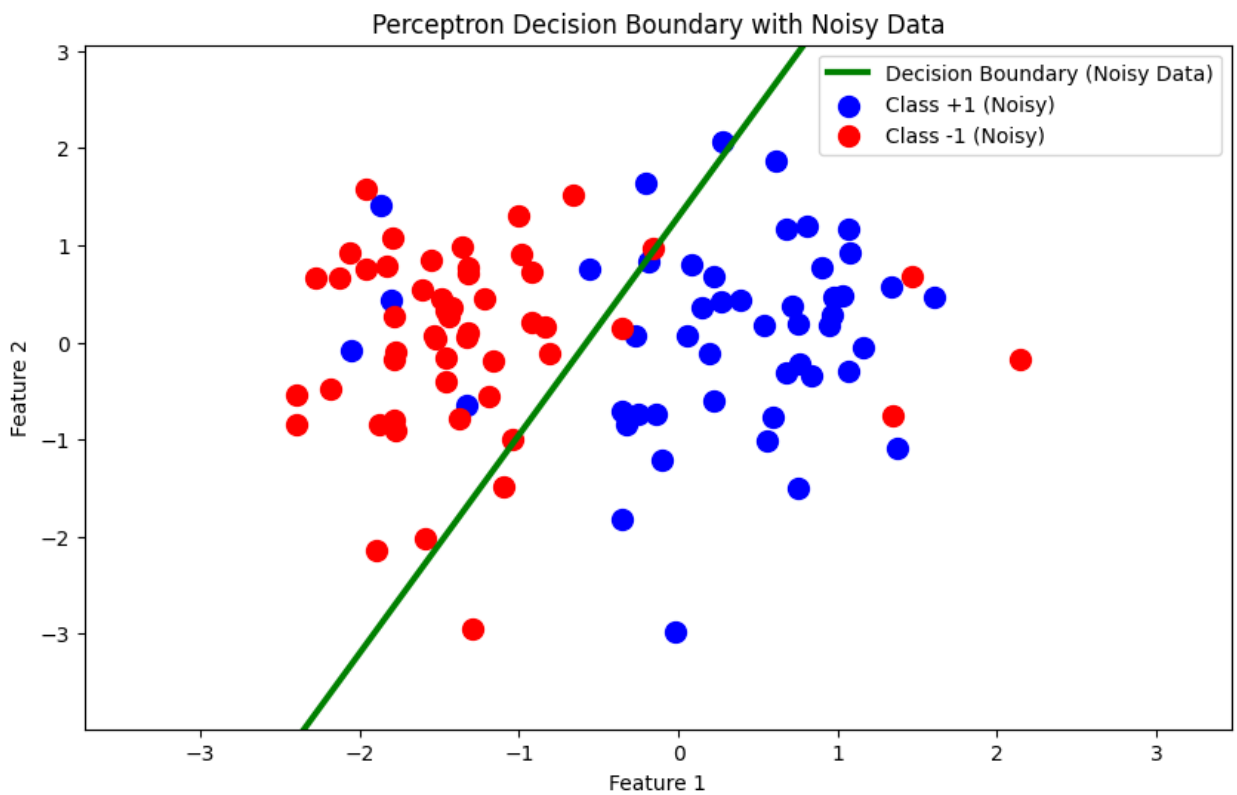
Now, let's train the perceptron on this noisy dataset and see how it performs.

```
# Train the perceptron using the noisy y labels
w_noisy, b_noisy = perceptron(X, y_noisy, iterations=100, lr=0.1)

print("Learned Weights (w) with Noise:", w_noisy)
print("Learned Bias (b) with Noise:", b_noisy)

# Plot decision boundary for noisy data
x_vals_noisy = np.linspace(X[:,0].min() - 1, X[:,0].max() + 1, 100)
y_vals_noisy = -(w_noisy[0]*x_vals_noisy + b_noisy)/w_noisy[1]
```

```
Learned Weights (w) with Noise: [ 0.17364262 -0.07732504]
Learned Bias (b) with Noise: 0.10000000000000003
```

```
plt.figure(figsize=(10,6))
plt.plot(x_vals_noisy, y_vals_noisy, color='green', linewidth=3,
label='Decision Boundary (Noisy Data)')

# Plot noisy data points
plt.scatter(X[y_noisy==1, 0], X[y_noisy==1, 1], color='blue',
label='Class +1 (Noisy)', s=100)
```

```
plt.scatter(X[y_noisy==-1, 0], X[y_noisy==-1, 1], color='red',
label='Class -1 (Noisy)', s=100)
plt.legend()
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Perceptron Decision Boundary with Noisy Data')
plt.ylim(X[:,1].min() - 1, X[:,1].max() + 1) # Adjust y-limits
dynamically
plt.show()
```




```
# Calculate and print accuracy for noisy data
pred_noisy = np.sign(np.dot(X,w_noisy) + b_noisy)
accuracy_noisy = np.mean(pred_noisy == y_noisy)
print("Accuracy with Noisy Data:", accuracy_noisy)
```

```
Accuracy with Noisy Data: 0.87
```